LESSON 7

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Number Display

Light up numbers with a 7-segment display and build an electronic dice.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How a **7-segment display** makes numbers out of 7 little lights
- How to use a **2D array**, a table of numbers in your code
- How to pick **random** numbers
- How to wait for a button to be **let go**

SESSION PLAN

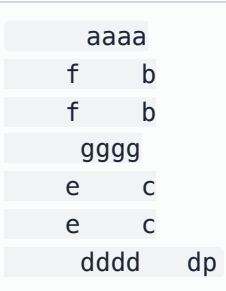
0:00	How does a 7-segment display work?
0:10	Wire up the display (take your time!)
0:45	Sketch 1: **L07_Segments** (test every light)
0:55	Short break
1:00	Sketch 2: **L07_Count** (numbers 0 to 9)
1:20	Sketch 3: **L07_Dice** (electronic dice)
1:40	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable, on your 2 breadboards
- ☐ 1-digit 7-segment display
- ☐ 8 resistors, 220 Ω (red-red-brown)
- ☐ 1 push button
- ☐ About 12 jumper wires

1. Seven little lights

Look closely at the display. The big **8** is really **7 separate LEDs** shaped like bars. There is also a tiny dot, the **decimal point**. Each bar has a letter name, from **a** to **g**:



Switch on the right bars and you get any number. For a **1**, light **b** and **c**. For an **8**, light all seven.

> **Big idea: one shared leg**
>
> Each of the 8 LEDs has its own **+** leg, but they all share a single **–** leg called the **common cathode** (COM). We connect COM to GND once. Then the ESP32 switches each bar on by setting its pin **HIGH**.

The display has 10 pins: 5 on the top and 5 on the bottom. Hold it with the **decimal point at the bottom right**. Pin 1 is then at the bottom left:

```
   pin: 10  9  8  7  6
        g   f COM a  b       (top row)
          +--------+
          |   8.   |
          +--------+
        e   d COM c  dp      (bottom row)
   pin: 1   2  3  4  5
```

2. Wire up the display

Push the display across the **middle gap** of a free part of a breadboard, with the decimal point at the bottom right. Every bar needs its own **220 Ω** resistor, just like a normal LED.

> **Neat wiring trick**
>
> Put each resistor from the display pin's row across to an empty row. Then run a jumper wire from that row to the ESP32. Check off each row as you go, because it's easy to lose your place!

✓	ESP32 pin	goes to	Display pin	Bar
☐	**23**	220 Ω resistor →	pin 7	a
☐	**22**	220 Ω resistor →	pin 6	b
☐	**21**	220 Ω resistor →	pin 4	c
☐	**19**	220 Ω resistor →	pin 2	d
☐	**18**	220 Ω resistor →	pin 1	e
☐	**17**	220 Ω resistor →	pin 9	f
☐	**16**	220 Ω resistor →	pin 10	g
☐	**4**	220 Ω resistor →	pin 5	dp
☐	**GND**	jumper wire →	pin 3 *or* pin 8	COM

Add the **button** for later: one leg to ESP32 pin **13**, the other leg to the **–** rail (GND).

✓	From	To
☐	button leg	ESP32 pin **13**
☐	other button leg (diagonally opposite)	**–** rail

3. Sketch 1: Test every segment

Before writing numbers, check that every bar works. Type this in and save it as **L07_Segments**.

Type this in → save it as **L07_Segments** 21 lines

```
// Lesson 7: Test every segment
// Lights segments a, b, c, d, e, f, g and dp one at a time.

const int segPins[8] = {23, 22, 21, 19, 18, 17, 16, 4};
const String names[8] = {"a", "b", "c", "d", "e", "f", "g", "dp"};

void setup() {
  Serial.begin(115200);
  for (int i = 0; i < 8; i++) {
    pinMode(segPins[i], OUTPUT);
  }
}

void loop() {
  for (int i = 0; i < 8; i++) {
    Serial.println(names[i]);
    digitalWrite(segPins[i], HIGH);
    delay(500);
    digitalWrite(segPins[i], LOW);
  }
}
```

Open the **Serial Monitor** (115200 baud). It tells you which bar *should* be lit. If the letter on screen doesn't match the bar that lights up, two wires are swapped.

Code	What it means
`const int segPins[8] = {23, 22, …};`	An **array**: a list of 8 pin numbers. `segPins[0]` is 23, `segPins[1]` is 22 …
`const String names[8] = {"a", …};`	A list of **words** (strings). `names[0]` is `"a"`
`for (int i = 0; i < 8; i++)`	Repeat for i = 0, 1, 2 … 7: one time for each bar

One bar stays dark?

Find its row in the wiring table and check that wire and resistor. If **nothing** lights, check the COM pin is connected to GND.

4. Sketch 2: Count from 0 to 9

Now we need a **table** that says which bars are on for each number. In code, a table is a **2D array**: an array of rows. Each row has 7 numbers, one for each bar from a to g. `1` means on and `0` means off.

File → New Sketch, type this in, and save it as **L07_Count**.

Type this in → save it as **L07_Count** 37 lines

```
// Lesson 7: Count from 0 to 9
// Each row of the table says which segments are on for one digit.

const int segPins[7] = {23, 22, 21, 19, 18, 17, 16};

const int digits[10][7] = {
// a  b  c  d  e  f  g
  {1, 1, 1, 1, 1, 1, 0},  // 0
  {0, 1, 1, 0, 0, 0, 0},  // 1
  {1, 1, 0, 1, 1, 0, 1},  // 2
  {1, 1, 1, 1, 0, 0, 1},  // 3
  {0, 1, 1, 0, 0, 1, 1},  // 4
  {1, 0, 1, 1, 0, 1, 1},  // 5
  {1, 0, 1, 1, 1, 1, 1},  // 6
  {1, 1, 1, 0, 0, 0, 0},  // 7
  {1, 1, 1, 1, 1, 1, 1},  // 8
  {1, 1, 1, 1, 0, 1, 1}   // 9
};

void showDigit(int n) {
  for (int s = 0; s < 7; s++) {
    digitalWrite(segPins[s], digits[n][s]);
  }
}

void setup() {
  for (int s = 0; s < 7; s++) {
    pinMode(segPins[s], OUTPUT);
  }
}

void loop() {
  for (int n = 0; n <= 9; n++) {
    showDigit(n);
    delay(1000);
  }
}
```

Code	What it means
`const int digits[10][7]`	A table with **10 rows** (digits 0–9) and **7 columns** (bars a–g)
`digits[n][s]`	Row **n**, column **s**. For example `digits[1][0]` is 0: bar **a** is off for the digit 1
`void showDigit(int n)`	Our own **function**. Give it a number and it switches on the right bars
`digitalWrite(segPins[s], digits[n][s]);`	`1` works just like `HIGH`, and `0` works just like `LOW`

> **Try it**
>
> 1. Look at the row for **7**. Which bars are on? Check it on the display.
> 2. Change `delay(1000)` to `delay(200)`. Super fast counting!
> 3. Change the loop so it only shows **even** numbers: 0, 2, 4, 6, 8. *Hint:* `n = n + 2`.

5. Sketch 3: Electronic dice

Time for a game! When you press the button, the display "rolls" through random numbers, slowing down, then stops on a number from 1 to 6.

File → Save As… your L07_Count sketch with the new name **L07_Dice**. Then change it to match this code. The big table stays the same, so you only need to type the new parts!

Type this in → save it as **L07_Dice** — 50 lines

```
// Lesson 7: Electronic dice
// Press the button on pin 13 to roll.

const int BUTTON_PIN = 13;
const int segPins[7] = {23, 22, 21, 19, 18, 17, 16};

const int digits[10][7] = {
// a  b  c  d  e  f  g
  {1, 1, 1, 1, 1, 1, 0},  // 0
  {0, 1, 1, 0, 0, 0, 0},  // 1
  {1, 1, 0, 1, 1, 0, 1},  // 2
  {1, 1, 1, 1, 0, 0, 1},  // 3
  {0, 1, 1, 0, 0, 1, 1},  // 4
  {1, 0, 1, 1, 0, 1, 1},  // 5
  {1, 0, 1, 1, 1, 1, 1},  // 6
  {1, 1, 1, 0, 0, 0, 0},  // 7
  {1, 1, 1, 1, 1, 1, 1},  // 8
  {1, 1, 1, 1, 0, 1, 1}   // 9
};

void showDigit(int n) {
  for (int s = 0; s < 7; s++) {
    digitalWrite(segPins[s], digits[n][s]);
```

```
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  for (int s = 0; s < 7; s++) {
    pinMode(segPins[s], OUTPUT);
  }
  showDigit(0);
}

void loop() {
  if (digitalRead(BUTTON_PIN) == LOW) {
    for (int i = 0; i < 12; i++) {  // rolling animation
      showDigit(random(1, 7));
      delay(40 + i * 15);
    }
    int roll = random(1, 7);
    showDigit(roll);
    Serial.print("You rolled a ");
    Serial.println(roll);
    while (digitalRead(BUTTON_PIN) == LOW) {
      delay(10);  // wait until the button is let go
    }
  }
}
```

Code	What it means
`random(1, 7)`	A random whole number from **1 up to 6**. The 7 itself is never chosen!
`delay(40 + i * 15);`	Each step waits a bit longer, so the roll **slows down** like a real dice
`while (digitalRead(BUTTON_PIN) == LOW)`	Keep waiting **while** the button is still held down

Big idea: really random

The ESP32 has a special circuit inside that makes **truly random** numbers from electrical noise. Most small microcontrollers can only fake it!

Try it

1. Roll the dice 20 times. Write down how many times you get a **6**.

---

2. What happens if you **delete** the `while` loop and hold the button down?

6. Challenges

Challenge 1: Countdown

Make the display count **down** from 9 to 0, then flash the 0 three times like a rocket launch. *Hint:* `for (int n = 9; n >= 0; n--)`.

Challenge 2: Letters!

A 7-segment display can also show **A b C d E F**. Add 6 more rows to the table (make it `[16][7]`), then count from 0 to F. Computer scientists call this **hexadecimal**.

Challenge 3: Dice with sound

Connect a **passive buzzer** (+ to pin **33**, – to GND). Make a click with `tone(33, 1500, 10);` on every step of the roll, and a happy beep at the end.

Bonus: Spinner

While the dice rolls, light just one outside bar at a time (a, b, c, d, e, f) so a light chases around the edge.

7. Quiz

Check what you learned

1. Which bars light up to make the number **4**?

---

2. In `digits[3][6]`, which row and which column is that?

---

3. What numbers can `random(1, 7)` give you?

Tidy up: unplug the USB cable. Leave the display and its resistors in place, because Lesson 9 uses the same wiring for the 4-digit display!